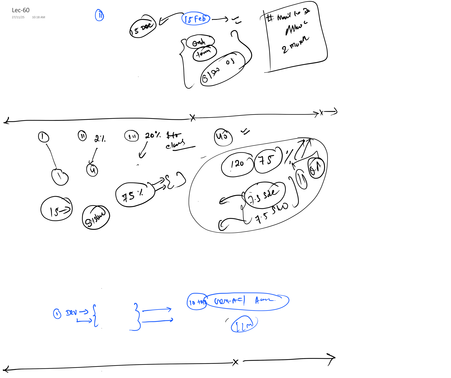
Lec-60


Bit-masking
mask
variable
Two's complement
~(1) = 0
~(0) = 1
a&b
n&(n-1) = 0
13<<1
13<<2
13<<3
13×2 = 26
13×2² = 104
13×8 = 104
a<<n → a×2ⁿ
1<<n → 2ⁿ
b&~b = 0
b<<1
13>>1 =
13>>2 =
13>>3 =
13/2 = 6
13/2² = 4
13/2³ = 1
a>>n = a/2ⁿ
n = n>>4
n&(n-1) = 1
XOR = a^a = 0
a^0 = a
a^b^c = b^a^c
= b^c^a
[2,3,1,3,1,2,]
Given a non-empty array of integers nums, every element appears twice except for one. Find that single one.
You must implement a solution with a linear runtime complexity and use only constant extra space.
n → n-1
n&(n-1)
Function Approach
XOR = XOR^A[i]
XOR = XOR^(1<<i)
y^0 = y^y^y
n = 84
bit
n = 01010108
while(n>>0)
n = n&(n-1)
cnt
A magic number is defined as a number which can be expressed as a power of 5 or sum of unique powers of 5. First few magic numbers are 5, 25, 30(5 + 25), 125, 130(125 + 5), ...
Write a function to find the nth Magic number.
Example:
Input: n = 2
Output: 25
Input: n = 5
Output: 130
5 25 30 125 130 150 155 625 630 650 655
5¹ 5² 5¹+5² 5³ 5³+5¹ 5³+5² 5³+5²+5¹ 5⁴ 5⁴+5¹ 5⁴+5² 5⁴+5²+5¹
mul = 5
while(n>0)
if(n&1) ans += mul
n = n>>1
mul = mul * 5